

WEEK 1 (Pravalika 2520030402)

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes. Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
UW PICO 5.89
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char cmd[100];
    pid_t pid;

    printf("Enter Linux command: ");
    scanf("%99[^\n]", cmd);

    pid = fork();

    if (pid < 0)
    {
        printf("fork() failed\n");
        exit(1);
    }

    else if (pid == 0)
    {
        // Child process
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execl("/bin/sh", "sh", "-c", cmd, NULL);

        // This executes only if exec fails
        printf("exec() failed\n");
        exit(1);
    }

    else
    {
        // Parent process
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed\n");
    }

    return 0;
}
```

Output:

```
Enter Linux command: ls
Parent Process
Parent PID: 537
Child PID: 538
Child Process
Child PID: 538
Parent PID: 537
2520030001_Skill      2520030402_Practical  addition.c      execsc2.c      exp2.c          p4.c
2520030002_Practical 2520030402_Skill    addition.c.save execsc3.c      filesystem.c     p5.c
2520030002_Skill     ForgeOS              dirl            execsc4.c      'gcc addition.c -o add' practical1.c
2520030003_Practical a.out               execsc.c        execsc5.c      p2.c            switch
2520030003_Skill     addition             execsc1.c       expl.c         p3.c            switch.c
Child process completed
```